
UNIT IV- PL/SQL PROGRAMMING

4.4 Cursors:

Implicit and Explicit Cursors

Declaring, Opening and Closing a Cursor

Fetching a Record from Cursor

Cursor for loops

Parameterized Cursors.

Prepared By
Mr. Mohammad Ali
Lecturer (Selection Grade)
M. H. Saboo Siddik Polytechnic, Mumbai

PL/SQL - Cursors

- Oracle creates a **memory area**, known as **context area**, for processing an SQL statement, which contains all information needed for processing the statement, for example, number of rows processed, etc.
- A **cursor is a pointer** to this **context area**.
- PL/SQL controls the context area through a cursor.
- A cursor holds the **rows (one or more)** returned by a SQL statement.
- The **set of rows** the cursor holds is referred to as the **active set**.
- We can name a cursor so that it could be referred to in a program to fetch and process the rows returned by the SQL statement, one at a time.
- There are two types of cursors:
 - **Implicit cursors**
 - **Explicit cursors**

Implicit Cursors

- Implicit cursors are automatically created by Oracle whenever an SQL statement is executed, when there is no explicit cursor for the statement.
- Programmers cannot control the implicit cursors and the information in it.
- Whenever a DML statement (INSERT, UPDATE and DELETE) is issued, an implicit cursor is associated with this statement.
- For INSERT operations, the cursor holds the data that needs to be inserted.
- For UPDATE and DELETE operations, the cursor identifies the rows that would be affected.
- In PL/SQL, we can refer to the most recent implicit cursor as the SQL cursor, which always has the attributes like %FOUND, %ISOPEN, %NOTFOUND, and %ROWCOUNT.
- The SQL cursor has additional attributes, %BULK_ROWCOUNT and %BULK_EXCEPTIONS, designed for use with the FORALL statement.
- The following table provides the description of the most used attributes:

Implicit Cursors

Attribute	Description
%FOUND	Returns TRUE if an INSERT, UPDATE, or DELETE statement affected one or more rows or a SELECT INTO statement returned one or more rows. Otherwise, it returns FALSE.
%NOTFOUND	The logical opposite of %FOUND. It returns TRUE if an INSERT, UPDATE, or DELETE statement affected no rows, or a SELECT INTO statement returned no rows. Otherwise, it returns FALSE.
%ISOPEN	Always returns FALSE for implicit cursors, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.
%ROWCOUNT	Returns the number of rows affected by an INSERT, UPDATE, or DELETE statement, or returned by a SELECT INTO statement.

Example - Implicit Cursors

- Any SQL cursor attribute will be accessed as **sql%attribute** name as shown below in the example.
- The following program would update the employee table and increase salary of each employee by 5000 and use the **SQL%ROWCOUNT** attribute to determine the number of rows affected:

```
DECLARE
    total_rows number(2);
BEGIN
    UPDATE employee
    SET sal = sal + 5000;
    IF sql%notfound THEN
        dbms_output.put_line('No Employee selected');
    ELSIF sql%found THEN
        total_rows := sql%rowcount;
        dbms_output.put_line( total_rows || ' Employees record updated ');
    END IF;
END;
/
```

Example - Implicit Cursors

- Before execution of above code the **Select * from employee;** produces the following result:

```
SQL> SELECT * FROM EMPLOYEE;
```

EMPNO	ENAME	DNAME	SALARY
101	ANAM	PERSONAL	65000
102	JUNAID	PERSONAL	55000
103	HABBAN	SALES	75000
104	CHAITANYA	SALES	50000
105	CHETAN	FINANCE	60000
106	ADITYA	SALES	65000

```
6 rows selected.
```

- When the above code is executed at SQL prompt, it produces the following result:

```
6 Employees record updated
```

```
PL/SQL procedure successfully completed.
```

If we check the records in employee table, we will find that the rows have been updated:

Select * from emp;

```
SQL> select * from employee;
```

EMPNO	ENAME	DNAME	SALARY
101	ANAM	PERSONAL	70000
102	JUNAID	PERSONAL	60000
103	HABBAN	SALES	80000
104	CHAITANYA	SALES	55000
105	CHETAN	FINANCE	65000
106	ADITYA	SALES	70000

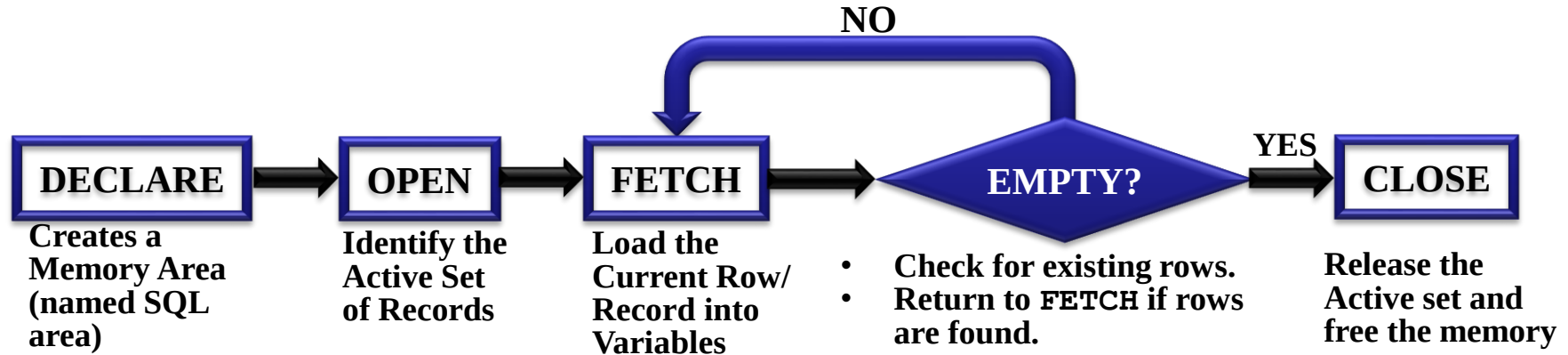
```
6 rows selected.
```

Explicit Cursors

- Explicit cursors are programmer defined cursors for gaining more control over the context area.
- An explicit cursor should be defined in the declaration section of the PL/SQL Block.
- It is created on a SELECT Statement.
- The syntax for creating an explicit cursor is :

CURSOR <cursor_name> IS <select_statement>;

Explicit Cursors



```
DECLARE
  e_id emp.empno%TYPE;
  e_name emp.ename%TYPE;
  e_sal emp.sal %TYPE;
  CURSOR cur_emp IS
    SELECT empno, ename, sal FROM emp;

BEGIN
  OPEN cur_emp;
  LOOP
    FETCH cur_emp INTO e_id, e_name, e_sal;
    EXIT WHEN cur_emp%NOTFOUND;
    dbms_output.put_line(e_id || ' ' || e_name || ' ' || e_sal);
  END LOOP;
  CLOSE cur_emp;
END;
/
```

Working with an explicit cursor involves four steps:

- Declaring the cursor for initializing in the memory
- Opening the cursor for allocating memory
- Fetching the cursor for retrieving data
- Closing the cursor to release allocated memory

Declaring the Cursor

- Declaring the cursor defines the cursor with a name and the associated **SELECT** statement.

For example:

CURSOR c_emp **IS**

SELECT empno, ename, sal **FROM** emp;

Opening the Cursor

- Opening the cursor allocates memory for the cursor and makes it ready for fetching the rows returned by the SQL statement into it.
- For example, we will open above-defined cursor as follows:

OPEN c_emp;

Fetching the Cursor

- Fetching the cursor involves accessing one row at a time.
- For example we will fetch rows from the above-opened cursor as follows:

FETCH c_emp **INTO** c_id, c_name, c_sal;

Closing the Cursor

- Closing the cursor means releasing the allocated memory.
- For example, we will close above-opened cursor as follows:

CLOSE c_emp;

Following is a complete example to illustrate the concepts of Explicit Cursor:

DECLARE

c_id emp.empno%type;

c_name emp.ename%type;

c_sal emp.sal%type;

c_rownum number(2);

total_rows number(2);

CURSOR c_emp **IS**

SELECT rownum, empno, ename, sal FROM emp;

BEGIN

dbms_output.put_line(CHR(9)||CHR(9)||CHR(9)||'EMPNO'||CHR(9)||'NAME'||CHR(9)||'SALARY');

dbms_output.put_line(CHR(9)||CHR(9)||CHR(9)||'====='||CHR(9)||'====='||CHR(9)||'====');

OPEN c_emp;

LOOP

FETCH c_emp **INTO** c_rownum, c_id, c_name, c_sal;

EXIT WHEN c_emp%**notfound**;

dbms_output.put('Employee No. '||c_rownum || CHR(9)||':'||CHR(9));

dbms_output.put_line(c_id || CHR(9) || c_name || CHR(9) || c_sal);

END LOOP;

total_rows := c_emp%**rowcount**;

-- dbms_output.new_line();

-- dbms_output.put_line("");

dbms_output.put_line(chr(10));

dbms_output.put_line(total_rows || ' Employees Record Selected.');

CLOSE c_emp;

END;

/

When the above code is executed at SQL prompt, it produces the following result:

	EMPNO	NAME	SALARY
	=====	=====	=====
Employee No. 1 :	7369	SMITH	800
Employee No. 2 :	7499	ALLEN	1600
Employee No. 3 :	7521	WARD	1250
Employee No. 4 :	7566	JONES	2975
Employee No. 5 :	7654	MARTIN	1250
Employee No. 6 :	7698	BLAKE	2850
Employee No. 7 :	7782	CLARK	2450
Employee No. 8 :	7788	SCOTT	3000
Employee No. 9 :	7839	KING	5000
Employee No. 10 :	7844	TURNER	1500
Employee No. 11 :	7876	ADAMS	1100
Employee No. 12 :	7900	JAMES	950
Employee No. 13 :	7902	FORD	3000
Employee No. 14 :	7934	MILLER	1300

14 Employees Record Selected.

PL/SQL procedure successfully completed.

Parameterized Cursors

- Parameterized cursors are static cursors that can accept passed-in parameter values when they are opened.
- PL/SQL Parameterized cursor pass the parameters into a cursor and use them in to query.
- PL/SQL Parameterized cursor define only datatype of parameter and not need to define it's length.
- Default values is assigned to the Cursor parameters. and scope of the parameters are locally.
- The following example includes a parameterized cursor. The cursor displays the name and salary of each employee in the EMP table whose salary is less than that specified by a passed-in parameter value.

```
DECLARE
  my_record emp%ROWTYPE;
  CURSOR c1 (max_sal NUMBER) IS
    SELECT * FROM emp WHERE sal < max_sal;
BEGIN
  OPEN c1(2000);
  LOOP
    FETCH c1 INTO my_record;
    EXIT WHEN c1%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Name = ' || my_record.ename || ',' || chr(9) || 'salary = '
      || my_record.sal);
  END LOOP;
  CLOSE c1;
END;
/
```

When the above code is executed at SQL prompt, it produces the following result:

```
SQL> ed c:\notes1\param_cur.sql
```

```
SQL> @ c:\notes1\param_cur.sql
```

```
Name = SMITH,      salary = 800
```

```
Name = ADAMS,      salary = 1100
```

```
Name = WARD,       salary = 1250
```

```
Name = MARTIN,     salary = 1250
```

```
Name = MILLER,     salary = 1300
```

```
Name = TURNER,     salary = 1500
```

```
Name = ALLEN,      salary = 1600
```

```
Name = JAMES,      salary = 1950
```

```
PL/SQL procedure successfully completed.
```